CO4_AT1_SIMATS_Compiler

View Only

← Back

QUESTIONS

- **Q1** Virtual Base Class for Battery Data — 10 marks
- **Q2** Scalable Circular-Economy Architecture — 10 marks
- **Q3** Hierarchical Inheritance for EVs — 10 marks
- **Q4** this Pointer in Carbon Monitoring — 10 marks
- **Q5** Dynamic Memory in E-Waste Management — 10 marks

Q5 Dynamic Memory in E-Waste Management

10 marks

An e-waste management system dynamically allocates recyclable products and releases them after processing. Analyze how new and delete work with constructors and destructors.

Your Code

```
#include <iostream>
using namespace std;

class RecyclableProduct {
private:
    int id;

public:
    RecyclableProduct(int i) {
        id = i;
        cout << "Constructor: Product " << id << " created." << endl;
    }

    ~RecyclableProduct() {
        cout << "Destructor: Product " << id << " released." << endl;
    }
```

Input

```
101
```

Output

Visible Test Cases

Test Case 1

QUESTIONS

Q1
Virtual Base Class for Battery Data
10 marks

Q2
Scalable Circular-Economy Architecture
10 marks

Q3
Hierarchical Inheritance for EVs
10 marks

Q4
this Pointer in Carbon Monitoring
10 marks

Q5
Dynamic Memory in E-Waste Management
10 marks

5/5 answered

Q2 Scalable Circular-Economy Architecture

10 marks

A circular-economy platform manages ReusableProduct, RecyclableProduct and RepairableProduct objects. Analyze how abstract classes, inheritance and pointers can create a scalable system.

Your Code

```cpp
#include <iostream>
using namespace std;

class Product {
public:
    virtual void process() = 0;
    virtual ~Product() {}
};

class ReusableProduct : public Product {
public:
    void process() override {
        cout << "Reusable Product: Product is reused." << endl;
    }
};

class RecyclableProduct : public Product {
public:
    void process() override {
        cout << "Recyclable Product: Product is recycled." << endl;
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

CO4_AT1_SIMATS_Compiler

View Only

QUESTIONS

- **Q1** Virtual Base Class for Battery Data — 10 marks
- **Q2** Scalable Circular-Economy Architecture — 10 marks
- **Q3** Hierarchical Inheritance for EVs — 10 marks
- **Q4** this Pointer in Carbon Monitoring — 10 marks
- **Q5** Dynamic Memory in E-Waste Management — 10 marks

Q3 Hierarchical Inheritance for EVs

10 marks

An EV company manages different vehicle types such as ElectricCar and HybridCar. Analyze how hierarchical inheritance can be used to reuse common vehicle and battery properties.

Your Code

```
#include <iostream>
using namespace std;

class Vehicle {
protected:
    string brand;
    int batteryCapacity;

public:
    void getVehicleData() {
        cin >> brand >> batteryCapacity;
    }

    void displayVehicleData() {
        cout << "Brand: " << brand << endl;
        cout << "Battery Capacity: " << batteryCapacity << " kWh" << en
```

Input

```
Tesla 75
Toyota 50
```

Output

Visible Test Cases

Test Case 1

QUESTIONS

Q1
Virtual Base Class for Battery Data
10 marks

Q2
Scalable Circular-Economy Architecture
10 marks

Q3
Hierarchical Inheritance for EVs
10 marks

Q4
this Pointer in Carbon Monitoring
10 marks

Q5
Dynamic Memory in E-Waste Management
10 marks

Q4 this Pointer in Carbon Monitoring

A carbon-monitoring system has a local variable and an instance variable with the same name. Analyze how the this pointer resolves the conflict.

Your Code

```cpp
#include <iostream>
using namespace std;

class CarbonMonitor {
private:
    int carbon;

public:
    void setCarbon(int carbon) {
        this->carbon = carbon;
    }

    void display() {
        cout << "Carbon Emission: " << carbon << " kg" << endl;
    }
};
```

Input

```
120
```

Output

Visible Test Cases

Test Case 1

CO4_AT1_SIMATS_Compiler

View Only

Back

QUESTIONS

- **Q1** Virtual Base Class for Battery Data — 10 marks
- **Q2** Scalable Circular-Economy Architecture — 10 marks
- **Q3** Hierarchical Inheritance for EVs — 10 marks
- **Q4** this Pointer in Carbon Monitoring — 10 marks
- **Q5** Dynamic Memory in E-Waste Management — 10 marks

Q5 Dynamic Memory in E-Waste Management

10 marks

An e-waste management system dynamically allocates recyclable products and releases them after processing. Analyze how new and delete work with constructors and destructors.

Your Code

```
#include <iostream>
using namespace std;

class RecyclableProduct {
private:
    int id;

public:
    RecyclableProduct(int i) {
        id = i;
        cout << "Constructor: Product " << id << " created." << endl;
    }

    ~RecyclableProduct() {
        cout << "Destructor: Product " << id << " released." << endl;
    }
```

Input

```
101
```

Output

Visible Test Cases

Test Case 1